

Phase 6C · Stand up the app on AWS

OWNER · ~2 HRS

Phases 1 to 4 built the EMR's half of the boundary. This phase builds the app's half: a **second, independent stack beside OpenEMR**, in the same BAA account. Nothing here changes the EMR — `gfc-emr-web`, `gfc-emr-db`, `gfc-openemr-db`, `gfc-openemr` and the `emr` DNS record all stay exactly as they are.

The steps mirror Phases 1 to 4 almost exactly. Different names, and Postgres instead of MySQL — the app needs Postgres, so sharing OpenEMR's database was never an option.

Why the app does not run on the OpenEMR box

That instance is a `t3.small` already carrying OpenEMR and its MySQL connections. Adding a Node process makes both slower, and worse, ties their fate together — a restart for one takes the other down. Bethel should not lose charting because the app needed a bounce. Separate instances, separate failure domains.

What this runs on, and what it adds

ITEM	VALUE
Application	GFC Care Platform, Node 22 in Docker, built from the <code>Dockerfile</code> in the repo
Host	EC2 <code>t3.small</code> , Ubuntu 24.04, 20 GB encrypted <code>gp3</code> , instance name <code>gfc-app</code> , Elastic IP
Database	RDS PostgreSQL 16 , <code>db.t4g.micro</code> , encrypted, 20 GB <code>gp3</code> autoscaling to 100 GB, 30-day automated backups, identifier <code>gfc-app-db</code>
Address	<code>https://app.godwinsfamilycarellc.com</code> (Let's Encrypt, issued by Caddy)
TLS	Caddy reverse proxy in the same compose file. An ALB does the same job for about \$16/month; at this scale Caddy is the better trade
Region	<code>us-east-1</code> , same as the EMR. Everything in one region, always
Added run cost	About \$30 to \$35 per month. Raise the budget alarm from \$80 to about \$140 to cover both stacks
Documents	Unchanged — consents and signed PDFs stay in HIPAA Google Drive under the Workspace BAA

6C.1 · Security groups

10 MIN

- `gfc-app-web`, "GFC app web server". Inbound: HTTPS 443 from Anywhere-IPv4; HTTP 80 from Anywhere-IPv4 (Let's Encrypt's HTTP-01 challenge needs it); SSH 22 from My IP; SSH 22 from `18.206.107.24/29` so the browser terminal can reach the box. Outbound default.
- `gfc-app-db`, "GFC app database". Inbound: PostgreSQL 5432 with source `gfc-app-web`. Nothing else. The database is never exposed to the internet.

6C.2 · The database (RDS Postgres)

20 MIN + 10 MIN WAIT

- 1 RDS → Create database → Standard create → **PostgreSQL** (latest 16.x) → template Production → Single DB instance.
- 2 Identifier gfc-app-db. Master username appadmin. Master password generated into the password manager as APP-DB-MASTER.
- 3 db.t4g.micro. Storage 20 GB gp3, autoscaling on, max 100 GB.
- 4 Connectivity: do not connect to an EC2 resource; default VPC; public access **No**; security group gfc-app-db only.
- 5 Additional configuration: **leave "Initial database name" blank**. The app creates its own schema on first boot; pre-creating it causes the first migration to fail. Backup retention 30 days, **backup window off clinic hours**. Encryption enabled. Deletion protection on.
- 6 When *Available*, copy the endpoint to the worksheet. It is the host portion of DATABASE_URL.

6C.3 · The app server and a permanent address

20 MIN

- 1 EC2 → Launch instance. Name gfc-app. Ubuntu Server 24.04 LTS x86. t3.small. Key pair gfc-emr-key (reuse; break-glass only).
- 2 Network settings → Edit: auto-assign public IP Enable; Firewall → select existing → gfc-app-web.
- 3 Storage 20 GB gp3 → Advanced → **Encrypted: Yes** (default KMS key). Launch.
- 4 **Permanent address:** EC2 → Elastic IPs → Allocate → Associate → instance gfc-app. Copy it to the worksheet. Without this the address changes on every restart and the domain breaks.

6C.4 · Point the domain at it

10 MIN + PROPAGATION

- 1 Log in wherever godwinsfamilycarellc.com's DNS is managed — the same place the emr record was added in Phase 4.
- 2 Add a record: Type **A** · Host/Name app · Value: the Elastic IP · TTL default (lower it to 300 the day before cutover).
- 3 Verify at dnschecker.org before continuing: look up app.godwinsfamilycarellc.com (type A). When it returns the Elastic IP from most locations, proceed.

Do not start 6C.5 until DNS resolves

Caddy requests the HTTPS certificate on first start and the certificate authority verifies the domain by visiting it. If DNS is not pointing at the server yet, issuance fails and you repeat the step.

6C.5 · Install and run the app

30 MIN

Everything below is typed in the browser terminal: **EC2** → **Instances** → **gfc-app** → **Connect** → **EC2 Instance Connect**. Same as Phase 5. Nothing touches your Mac.

Prepare the server. Paste each block, press Enter, wait for the prompt.

```
sudo apt-get update && sudo apt-get -y upgrade
sudo apt-get -y install docker.io docker-compose-v2 git
sudo systemctl enable --now docker
sudo usermod -aG docker ubuntu && newgrp docker
```

Get the code. The repository is private, so use a deploy key or a fine-grained token with Contents: read.

```
sudo mkdir -p /opt/gfc && sudo chown ubuntu:ubuntu /opt/gfc
cd /opt/gfc
git clone https://github.com/bume1/Godwins-Family-Care.git app
cd app
```

Write the Caddy config. Two lines. Caddy obtains and renews the certificate on its own.

```
cat > /opt/gfc/app/Caddyfile <<'EOF'
app.godwinsfamilycarellc.com {
    reverse_proxy app:3000
}
EOF
```

Write the compose file.

```
cat > /opt/gfc/app/compose.yaml <<'EOF'
services:
  app:
    build: .
    restart: always
    env_file: /opt/gfc/app.env
    expose: ["3000"]

  caddy:
    image: caddy:2-alpine
    restart: always
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: [app]

volumes:
  caddy_data:
  caddy_config:
EOF
```

Generate the two new secrets. Everything else comes from the worksheet.

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))" # JWT_SECRET
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))" # EMR_TOKEN_ENCRYPTION_KEY
```

Write the environment file. Kept outside the repo directory so a git pull can never touch it, root-owned and unreadable by anyone else.

```
umask 077
cat > /opt/gfc/app.env <<'EOF'
NODE_ENV=production
DATA_STORE=postgres
DATABASE_URL=postgres://appadmin:<APP-DB-MASTER>@<RDS-ENDPOINT>:5432/gfc?sslmode=verify-full
JWT_SECRET=<generated above>
EMR_TOKEN_ENCRYPTION_KEY=<generated above>
OPENEMR_BASE_URL=https://emr.godwinsfamilycarellc.com
OPENEMR_SITE=default
OPENEMR_CLIENT_ID=<v4 client id>
OPENEMR_CLIENT_SECRET=<v4 client secret>
OPENEMR_REDIRECT_URI=https://app.godwinsfamilycarellc.com/oauth/callback
EOF
sudo chown root:root /opt/gfc/app.env && sudo chmod 600 /opt/gfc/app.env
```

Secrets: interim versus intended

Part VI control C-9 specifies AWS Secrets Manager. A root-owned 600 file on an encrypted volume inside the boundary is a defensible interim and is what the commands above produce. Secrets Manager with an instance IAM role is the stronger posture and the intended end state. If you go live on the file, record it as a known deviation with a review date.

Launch.

```
cd /opt/gfc/app
docker compose up -d --build
docker compose logs -f
```

The first build takes a few minutes. Watch for the boot line naming the adapter: Data store: postgres (production, inside the BAA boundary). Press Ctrl+C to leave the log view; the server keeps running.

If it refuses to boot, read the message — that is the guard working

The app deliberately will not start in production unless DATA_STORE=postgres, DATABASE_URL, JWT_SECRET and EMR_TOKEN_ENCRYPTION_KEY are all set, and it refuses MFA_ENFORCE=false outright. Every refusal names its own reason. A refusal is the control doing its job, not a failure of the install.

6C.6 · Verify, back up, and hand off to Session 5

15 MIN

- 1 From your own browser: <https://app.godwinsfamilycarellc.com/healthz>. Expected: {"ok":true,"store":"postgres","production":true}, a valid padlock, no warning. This is the line the Session 5 runbook checks as preflight P1.
- 2 **Snapshots.** EC2 → Elastic Block Store → Lifecycle Manager → create an EBS snapshot policy targeting the gfc-app volume, daily, 14-day retention. Same pattern as Phase 6 for the EMR box.
- 3 **Cost alarm.** Billing → Budgets → raise the monthly budget from \$80 to about \$140 to cover both stacks.
- 4 **Logs.** CloudWatch → create a log group for the app with a retention period set. Application logs are PII-scrubbed by Session 5, but keep them inside the account regardless.
- 5 **Monthly patch routine** — add to the Part VI calendar alongside the EMR routine: `cd /opt/gfc/app && git pull && docker compose up -d --build`.

What this phase does and does not finish

6C satisfies preflight **P1** (in-boundary host with TLS) and **P2** (encrypted RDS) of the Session 5 cutover runbook. It does **not** move any data — the store is empty and the app is not yet serving anyone. The migration from Replit KV, the DNS switch for users, the Replit decommission and the go/no-go all belong to the runbook's §4 sequence, which starts the moment `/healthz` answers correctly.

Adding features after the move. Development does not change. Claude Code edits and merges as before; the only difference is that deployment becomes one deliberate command on the box instead of an automatic Replit redeploy. Build features after the transition rather than before — anything built on Replit has to be retested here anyway, and cannot be exercised against real data until it lives inside the boundary.

Phase 6D · Cut over to the app on AWS

OWNER · HALF A DAY

6C built an empty app on AWS. 6D moves the real work onto it and closes the old doors behind you. This is the phase that lifts **TEST DATA ONLY** — nothing before it may hold a real patient.

The full technical detail lives in `docs/GFC_Session5_Cutover_Runbook.md`. This is the same sequence in the order you will actually do it. Where the runbook and this phase differ, this phase is the one to follow — it is written for the person clicking.

The one rule that governs the whole day

Prove the new path works before you close the old one. Every failure mode in this phase comes from disabling something before its replacement was verified. If you follow the numbered order you cannot lock yourself out; if you jump ahead to the tidy-up steps you can.

6D.1 · A week before

REHEARSAL

- 1 Rehearse the whole thing on a scratch copy: a second RDS restored from a snapshot and a scratch app host. Run the export, the dry-run migration, the real migration, and the verify pass. Fix anything there, not on the day.
- 2 Have one clinician complete the OpenEMR sign-in through the scratch app. This is the step nobody can do for you — it needs a human at OpenEMR's login page.
- 3 Lower the DNS TTL for `app.godwinsfamilycarellc.com` to 300 seconds so the switch is quick.

6D.2 · On the day · snapshots first

ROLLBACK ARTEFACTS

Nothing else happens until these three exist. They are the way back.

- 1 RDS manual snapshot of the app database, named `gfc-app-precutover-<date>`.
- 2 EBS snapshot of the OpenEMR volume.
- 3 Export the current Replit data: `node scripts/export_kv_snapshot.js`. Store the file in S3 in the BAA account with encryption, or on the app host's encrypted volume. **Never on a laptop.**

6D.3 · Migrate the data

30 MIN

Three commands, in order. The first two are safe; the third only confirms.

```
node scripts/migrate_kv_to_postgres.js --from-file <snapshot> --dry-run
node scripts/migrate_kv_to_postgres.js --from-file <snapshot>
node scripts/migrate_kv_to_postgres.js --from-file <snapshot> --verify-only
```

The dry run copies nothing. It lists every collection it found with row counts. If it stops with `UnhandledCollectionError`, that is the safety net: a collection exists that the migration does not know how to handle. Add the handler and rerun. **Nothing has been copied at that point.**

The real run must end `DONE ... VERIFY: n/n matched`. Running it a second time must report copied `0` — that is how you know it is safe to repeat.

6D.4 · Go / no-go

DECIDE BEFORE DNS

Every line true, or you stop. Up to this point **nothing user-facing has changed** and there is nothing to undo.

CHECK	WHAT GOOD LOOKS LIKE
Health	/healthz answers store: postgres, production: true
Migration	Report says ok, zero conflicts, no mismatches, and every PHI collection is listed
Repeatability	Verify-only passes; a second run copied 0
Acceptance	verify_session5.js passes on the new host — MFA, sessions, idle logout, audit, EMR handshake
Clinician	One clinician signed in to OpenEMR through the app, and a test note read back under her username
Rollback	Both snapshots and the export exist and are dated today
Paperwork	BAAs on file for AWS and Google Workspace

6D.5 · Flip, and everyone signs in again

20 MIN

- 1 Point `app.godwinsfamilycarellc.com` at the new server.
- 2 **Every existing login is dead, by design.** Everyone signs in fresh. Admin, clinical and case-manager users enrol MFA at that first login and are shown ten recovery codes **once** — they go into the password manager that day.
- 3 Each clinician clicks **Sign in to OpenEMR** in the workspace. From here on their writes carry their own name, not the service account's.
- 4 Run the two verification scripts against the live host and keep the output.

6D.6 · Close the old doors

IN THIS ORDER

Only after 6D.5 step 3 has actually succeeded for a real clinician.

- 1 **OpenEMR:** Administration → Config → Connectors → **Enable OAuth2 Password Grant** → **OFF**.
- 2 **OpenEMR:** disable the gfc-app-api user. The app no longer authenticates as it.
- 3 **OpenEMR:** API Clients → disable the v2 and v3 clients, delete the never-enabled duplicate. Keep v4. The password grant does not validate the client secret, so every enabled client is a live credential — this is access control, not tidying.
- 4 **Replit:** stop the deployment and delete its secrets. Leave its data store read-only for 7 days, then delete it. The marketing site stays on Replit; it holds no patient data.

6D.7 · Purge the test data

BEFORE THE FIRST REAL PATIENT

This one is easy to skip and expensive to skip

Every acceptance run since Session 4.1 left named test records behind. In OpenEMR: delete TEST PatientOne and every other test patient, with their encounters, notes, orders and **fee-sheet charges**. In the app: the matching client records, visit logs, consent events and billing rows. A test encounter carrying a CPT code is one claim batch away from going out the door. Record what was purged and by whom.

6D.8 · If it goes wrong

ROLLBACK

WHEN	WHAT TO DO
Before DNS flipped	Nothing user-facing changed. Leave the old deployment running. Keep or discard the new database. No data to reconcile.
After DNS, same day	Point DNS back. Anything written on the new host since the flip exists only in the new database — export it and reconcile those collections before reopening writes. MFA enrolments do not carry back.
Database damaged	Restore the pre-cutover snapshot to a new instance, repoint the app, redeploy. This is the drill you rehearsed.
OpenEMR itself	Independent of the app — restore from the EBS snapshot per Phase 6A's rollback.

The decision to roll back is Bianca's. The trigger is any go/no-go line failing on the live system, or a clinician unable to chart.

6D.9 · Locking yourself out, and how not to

BREAK-GLASS

Bianca is currently the only person with full access to both systems. A lost phone must not lock the practice out of its own records.

- 1 **Before go-live:** create a second named super-admin in the app and enrol MFA on it. Its recovery codes go in the password manager as a separate entry, dated.
- 2 Store your own ten recovery codes the day you enrol. Regenerate when fewer than three remain.
- 3 Either admin can reset the other's MFA from the admin hub; the user re-enrols at next login and all their sessions are revoked.

- 4 If both are locked out, an operator with database access clears the MFA field directly — deliberately manual, and logged. Write down who did it, when and why.
- 5 OpenEMR's own MFA is separate and the app cannot reset it. Keep the OpenEMR admin recovery codes in the password manager too.

When TEST DATA ONLY lifts

After 6D.7, with every go/no-go line green and both verification scripts passing on the live system. It is a decision Bianca makes and records — not something a merge or a deploy does on its own. Session 12 closes the remaining compliance items: the audit-log interface, breach-notification procedure, and the 7-year retention and disposal policy.